

be associated with your commits. You can do this using the following commands:

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

Additionally, you can customise various aspects of GIT behaviour using the *git config* command — for example, setting up aliases for frequently used commands, specifying default text editors, and configuring merge and diff tools.

Creating a new repository

Once GIT is installed and configured, you're ready to create your first repository. Navigate to the directory where you want to initialise the repository and run the following command:

```
git init
```

This command creates a new GIT repository in the current directory, initialising the necessary metadata and data structures to track changes to your project files.

Alternatively, if you want to work with an existing repository hosted on a remote server (e.g., GitHub, GitLab), you can clone it using the *git clone* command:

```
git clone <repository_URL>
```

Replace *<repository_URL>* with the URL of the remote repository you want to clone.

With your repository initialised or cloned, you're now ready to start adding files, making changes, and committing your work.

Basic GIT commands

Now that you've set up your GIT environment and initialised your repository, it's time to familiarise yourself with some of the basic GIT commands. These commands will enable you to track changes to your project, manage branches, and collaborate with other developers effectively.

1. **git add**

The *git add* command is used to add changes from the working directory to the staging area, preparing them for the next commit. You can add specific files or directories, or use wildcards to add multiple files at once.

```
git add <file1> <file2> # Add specific files
git add .                # Add all changes in the current directory
git add *.txt            # Add all .txt files in the current directory
```

2. **git commit**

Once changes are staged, you can commit them to the

repository using the *git commit* command. Each commit should have a meaningful commit message that describes the changes made.

```
git commit -m "Commit message"
```

3. **git status**

The *git status* command provides information about the current state of the repository. It shows which files have been modified, staged, or untracked.

```
git status
```

4. **git log**

To view the commit history of the repository, you can use the *git log* command. This command displays a list of commits along with their commit messages, authors, timestamps, and commit IDs (SHA-1 hashes).

```
git log
```

5. **git branch**

Branching is a powerful feature of GIT that allows for parallel development. The *git branch* command is used to create, list, or delete branches in the repository.

```
git branch                # List all branches
git branch <branch_name> # Create a new branch
git branch -d <branch_name> # Delete a branch
```

6. **git merge**

The *git merge* command is used to integrate changes from one branch into another. It combines the changes made in the specified branch into the current branch.

```
git merge <branch_name>
```

7. **git pull and git push**

To synchronise your local repository with a remote repository, you can use the *git pull* and *git push* commands, respectively. *git pull* fetches changes from the remote repository and merges them into the current branch, while *git push* uploads local commits to the remote repository.

```
git pull origin <branch_name> # Pull changes from the
remote repository
git push origin <branch_name> # Push changes to the remote
repository
```

Mastering these basic GIT commands is essential for efficiently managing your project's version control.